

| Backend Engineer

010-9101-5429 | digle117@gmail.com | github.com/PreAgile | astro-paper-23v.pages.dev

📄📄📄

📄·📄·📄 📄📄 📄·📄📄 📄 📄📄📄 📄📄, 📄📄📄 📄📄, 📄📄 📄, 📄 📄, 📄 📄 📄📄 📄📄 📄 5📄 📄📄 📄📄📄📄. 📄📄 📄📄 📄 SaaS 📄📄 📄📄 **API** 📄 **20** 📄, 📄📄 📄📄 **100** 📄, 📄 📄 **12** 📄 📄📄 📄📄 📄 📄📄📄 📄📄 📄, 📄 📄📄 📄📄 ADR 📄 PR 📄📄 📄📄 📄📄 📄📄📄.

NestJS 📄 📄 📄📄📄 📄📄 📄 📄, 📄📄📄, 📄📄 DB 📄 📄 📄, 📄 📄📄, 📄 📄📄 📄 📄📄

Kotlin/Spring/JPA/Kafka/Resilience4j 📄📄📄 📄📄📄 JVM 📄📄 📄📄 📄📄📄 📄📄. 📄·📄📄📄📄📄 📄📄 📄📄 📄📄 📄 📄📄 📄📄. **Kotlin** 📄📄 📄📄📄 **Kotest** 📄 **organization** 📄 📄📄 **6** **PR** 📄 📄📄, **LINE Armeria** 📄 MDC export 📄 PR 📄, **Spring Batch** 📄 4📄 📄 📄📄📄 📄 chunk scanning 📄 📄 PR 📄 📄📄📄.

📄📄 📄📄 📄📄

📄📄	Before → After	📄📄
📄📄 📄 📄	📄 800 → 90 📄 (88.75% ↓)	📄 IP 📄 📄📄, 📄 📄📄
📄📄 📄📄	70% → 98%	📄📄 📄 📄 📄📄
📄📄 📄📄	99.2% (📄 6📄)	📄 📄 📄📄📄 FIFO + lease TTL
📄📄 📄📄 📄	0 / 6 📄	Idempotency-Key + 📄 📄 📄
Akamai 📄📄 📄📄	77.8% → 100% (18/18 iter)	Referrer Warming 📄 📄
📄📄 📄📄📄 webhook race	📄·📄📄 📄📄 📄	📄📄 📄 📄📄 📄 📄 📄📄
📄📄📄	Kotest 6 PR 📄 (org 📄) / Armeria 1 PR 📄 📄 / Spring Batch · Spring Cloud Gateway 📄 📄 + PR 📄 📄	GitHub PR 📄

📄📄📄 📄📄

JVM 📄📄 OSS 📄 📄📄 📄 — 📄 **6** · 📄 📄 **1** · 📄📄 📄 + **PR** 📄 📄.

Kotest — Kotlin 📄📄 📄📄📄 (4.7k stars)

2026.03 📄📄📄📄 sksamuel 📄 📄📄 📄📄 **organization** 📄 📄, 📄 📄 6 PR 📄.

- [#5905](https://github.com/kotest/kotest/pull/5905) — 📄📄 📄 Test Metadata Public API 📄📄 📄 (+355/-16, 5 📄 16h 📄 📄)
- [#5835](https://github.com/kotest/kotest/pull/5835) — 📄📄 data class 📄 📄 diff
- [#5807](https://github.com/kotest/kotest/pull/5807) — JSON Schema anyOf/oneOf 📄 📄

- [#5795](https://github.com/kotest/kotest/pull/5795) — JSON Matchers 测试 Json 数据
- [#5789](https://github.com/kotest/kotest/pull/5789) — @OnlyInputTypes 限制输入类型
- [#5756](https://github.com/kotest/kotest/pull/5756) — shouldHaveSingleElement 测试集合元素

- [#6683](https://github.com/line/armeria/pull/6683) — RequestContextExporter BuiltInProperty ☐☐ + Logback MDC export ☐☐☐ ☐☐ (+339/-5, ☐☐☐☐☐ 1/2 ☐☐, ☐☐☐ ☐☐ ☐☐ ☐)

- [#3946](https://github.com/spring-projects/spring-batch/issues/3946) — fault-tolerant step chunk scanning 실패 시 4초 대기 (문서화 "documentation issue" 없음)
- [#4478](https://github.com/spring-projects/spring-batch/issues/4478) — conditional flow decider order 문제 (문서화 "this is a bug" 있음)

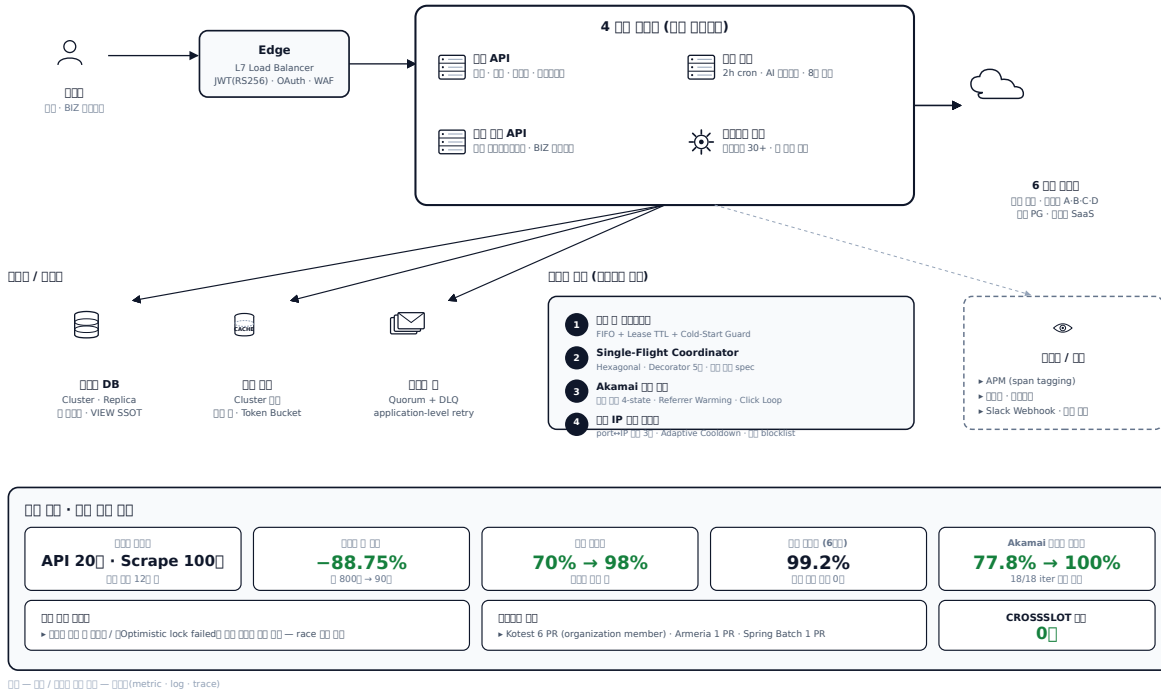
- [#3311](https://github.com/spring-cloud/spring-cloud-gateway/issues/3311) — Apache HttpClient5 100% 100% 100% (100% 100% 100%, 100% 100%, 5.1.x 100% 100%)
- [#4024](https://github.com/spring-cloud/spring-cloud-gateway/issues/4024) — API Versioning Predicates 100% 100% 100% (가 100% 100% 100%)

1111

項目	概要	詳細
API 仕様	API仕様書 (BIZ 側 ↔ 側)	REST API 137 controllers / 152 項目
運用体制	24時間 cron 監視・AI 監視・Active-Active 8 台	5,700+ 人 / 13,000+ 人
API 連携	API 連携・監視・AI 監視・BIZ 連携	Hash + ZSet
運用体制	6 台 監視・IP 監視・Akamai 監視・IP 監視	worker 30+

A1. 00 00 00 SaaS — 0000 00000

4 00 0000 · 6 00 0000 00 · 00 00000 · 00 00 0



A1. 00 0000 00000

0000 00·00(A) → 00 00 0000·00 0000(B) → 00 0000 00·0 00 00(C) → AI 000000·00 0000(D) 0 0000 15 0 0000 00 → 00 → 00 0000 000000.

0000 A. 00·00

00 1. 00 webhook 40 0000 — 00 0 (ABA-safe) · DB UNIQUE · 00000 · | 0 00 2025.09 ~ 00 0

00 — 00 PG webhook 0 00 00 00000 00 00000 0000 50 00 000000. ① PG 00 retry(5xx/timeout) ② 00 00 schedule 10 00 0 fallback retry 0 PAID/FAILED 가 00 00 00 ③ 00 00 0 0000 ④ 00 00 00 00 ⑤ Load Balancer keep-alive 0000. 00 INSERT 0 00 row 00 00 → 00 00 → 0000 20 00 → 00000 00 00 → 00 0 00 00000 00000. 00 000000 000000 in-memory dedupe 가 가, ORM 0 findOne → save 0000 read-modify-write 0000 race window 가 000000. 00 00 0 000000 TTL 00 0 00 0000 00 00 0000 ABA 00 (Martin Kleppmann 0 Redlock 00000 0000 hazard) 가 0000 0 00, DB UNIQUE 000000 PG retry 0000 10000 0000 0 0000, 0000000000 00 INSERT race 0 00 0 00000.

00 — 00 00000 00 race 0 00 0 0000 00000 40 0000 0 000000 00000 00000000. ① 00 00 SETNX EX + 00 00 lock value = ABA-safe distributed lock (Stripe 0 Idempotency Key 00000 00 0, lock value 0 ownership 00) ② DB UNIQUE on imp_uid + merchant_uid + status 00 0 = 00 0000 00(100 dedupe window) ③ 00000 READY → PAID/FAILED/CANCELLED × action_type 00 ④ Lua eval 0 GET → 00 → DEL 00 00, 0000 00 00. 0 00000 0 00 0000 00 0000 00000000 — 00 0000 00 0000 fail-open 0000 DB UNIQUE 가 00 00, 00 00(PAID 0 FAILED 00) 0 00 00 status 가 00000 00 row 0

A2. webhook 4 — DB . . .



- 락 락 락 락 락 락 락 락 락 race 가 락 락 락 락 락 락 락 락.
- 락 락 락 락 Slack 락 락 — 락 락 락 락 락.
- APM span tagging 락 락 락 락(amount_mismatch, coupon_select) 락 락.
- 락 락 helper(delIfValueMatches)가 락 락 8 락 락 락(promotional upgrade · change plan · verify code · 락 락) 락 락 helper 락 락.

4

2. **reconciliation** — **schedule** · **admin override orphan** | 2025.10 ~

— 1 ~ 4 **eventual inconsistency** 3 . ① **schedule** — PG **schedule** DB **cancel** ② — **SCHEDULED billing** 2 **admin override orphan** — 1 **schedule** **active** . **PG API** **DB** **SCHEDULED** **PG** **schedule_status** **DB** **billing** , **PG** **schedule** **eventual consistency** .

— $K \ll N$ **4-way** **cron** + **DB** **customer** **K** **PG** **schedule API** , **4-way** — **schedule 0** **DB** , **1** , **2** **schedule** **cancel** , **API** **Slack** . **admin override orphan** **SQL** . **cron** **SafeCron** (14) .

- 3 **inconsistency(zombie_schedule / duplicate_billing_key / admin_override_orphan)** .
- CS** — 09:10 KST **Slack** .
- K** (**active S**) — **PG** **rate-limit** 5 **cron** .
- SQL** **admin override** .

JVM/Spring — **Spring Scheduler** + **ShedLock** (**cron**) · **JdbcTemplate** / **QueryDSL** · **Spring Retry** + **RateLimiter** **PG** · **ApplicationEventPublisher** **Slack** .

3. 3월 3일 - 3월 3일 · schedule fallback 3월 3일 idempotency 2025.11 ~ 3월 3일

3월 3일 - 3월 3일 3월 3일 3월 3일. 3월 3일 schedule 3월 3일 3월 3일 amount 3월 3일 3월 3일 3월 3일 silent failure 가 가 3월 3일. schedule 3월 3일 3월 3일 A 3월 3일 50% 3월 3일 amount 3월 3일 3월 3일 3월 3일 A 가 3월 3일/3월 3일 3월 3일 PG 3월 3일 amount 3월 3일 3월 3일 → 3월 3일 3월 3일. 가 3월 3일 3월 3일 API 가 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일. 3월 3일 3월 3일 + 3월 3일 3월 3일 plan + 3월 3일 user 3월 3일 3월 3일 3월 3일 API 3월 3일 3월 3일 update 3월 3일 3월 3일 3월 3일 3월 3일.

3월 3일 - 3월 3일 idempotency 3월 3일: ① 3월 3일 - createUserCoupon 3월 3일 (user, coupon) 3월 3일 row 3월 3일, 3월 3일 row 3월 3일 3월 3일(idempotent return) ② 3월 3일 - 3월 3일 ORM QueryRunner 3월 3일 3월 3일 3월 3일 (startTransaction / commit / rollback / release) 3월 3일 delete + add + update 3월 3일 3월 3일 ③

schedule fallback 3월 3일 - schedule 가 3월 3일 3월 3일 user-coupons 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 → 3월 3일 3월 3일 schedule amount + coupon_id 3월 3일. 가 3월 3일 3월 3일 schedule·change_plan·promotional upgrade 3월 3일 3월 3일 3월 3일 DRY 3월 3일.

3월 3일

- 3월 3일 3월 3일 schedule fallback 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일.
- 3월 3일 3월 3일 3월 3일 0 - UNIQUE(user_id, coupon_id) + findOne 3월 3일 3월 3일 3월 3일.
- 3월 3일 3월 3일 update 3월 3일 0.

JVM/Spring 3월 3일 - JPA @OneToMany + @Version 3월 3일 · Spring TransactionTemplate 3월 3일 3월 3일 3월 3일 · Hibernate @PostLoad 3월 3일 schedule fallback 3월 3일 3월 3일 · Bean Validation @AssertTrue 3월 3일 amount 가 3월 3일

□□□ **B.** □□ □□ □□□ · □□ □□□ □□

4. 락 락 락 — 락 X-lock + | 락 락 락 2025.12 ~ 락 락

— 락 락 ~ 락 락 락 락 1 = 1 락 = 1~6 락 락 락 락 락 락. 락 락 INSERT 락 1,000 락 락 락, 락 (platform, platform_id) 가 락 락 락(락 락) 가 락 락 락 findOne → save race 락 락 row 가 락. ORM 락 락 REPEATABLE READ + UNIQUE 락 락 race window 가 락 ER_DUP_ENTRY 락 락 락 락 락, 1,000 락 락 락 락 락 락 락 락 락 락 락/가 row 락 락 락.

— `SELECT ... FOR UPDATE` 락 + 락 락 락 락 락 락 락. 락 락(groupKey = email + business_number) 락 락 @Transactional() 락 락 락 = 락 락. 락 락 (platform, platform_id) 락 X-lock 락, 락 락 락 락 락 marking. 락 락 락 findOne 락 + 락 락 update(attempt_count + 1) 락 race-free 락. 락 PENDING → VALIDATED → REGISTERING → REGISTERED | REGISTER_FAILED 가 락 락 last_error · last_attempted_at · attempt_count 락 row 가 락 락 락 락 락 락 락.

락

- 1,000+ 락 락 락, 락 락/가 row 락 락 락.
- 락 platform_id 락 락 락 row 0.
- 「 1,000 락 950 락, 50 락 락 50 락 락 — 락 락 락.

JVM/Spring 락 — JPA LockModeType.PESSIMISTIC_WRITE · Spring

@Transactional(propagation=REQUIRES_NEW) 락 락 락 락 · JpaSpecificationExecutor 락 락 락 · findAllById + saveAll batch.

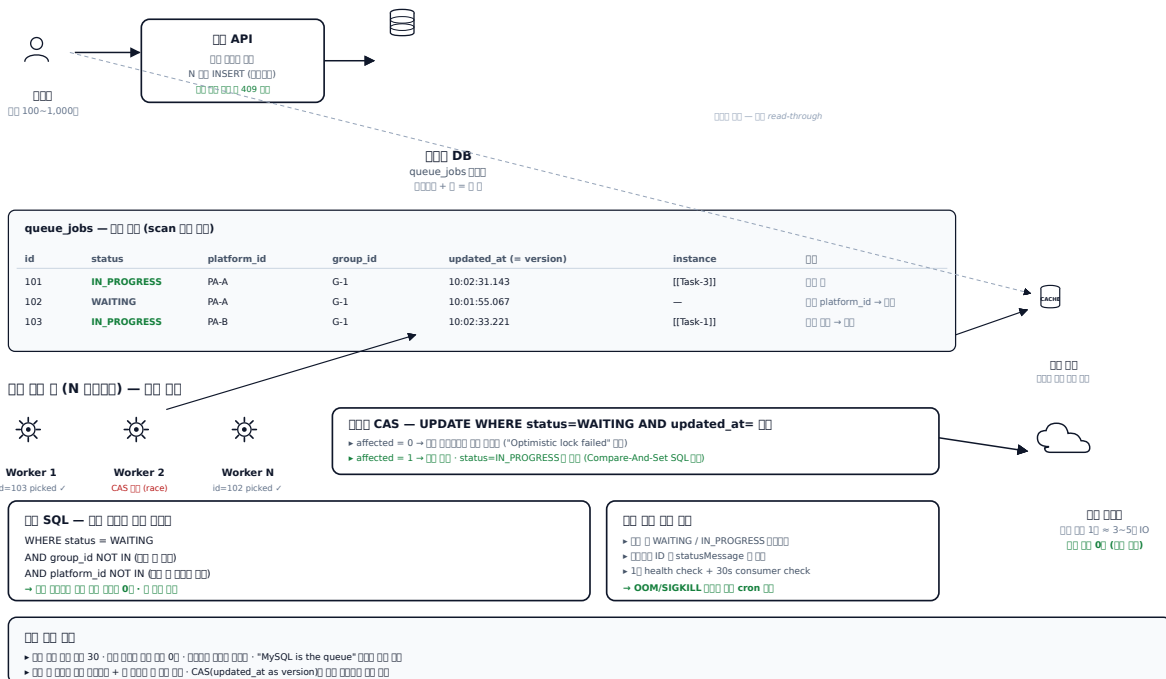
5. DB CAS | 2025.10 ~

100~1,000 IO — HTTP Load Balancer 5 in-memory queue worker 2 stateful (fire-and-forget),

DB ORM = CAS(Compare-And-Set) SQL updated_at version UPDATE WHERE id=? AND status=WAITING AND updated_at= affected=0 SQL platform_id NOT IN (status=WAITING/IN_PROGRESS + ID) statusMessage acquireProcessingLock

B1. DB CAS

MySQL is the queue, CAS



B1. RDS Queue + CAS

30

- 30, +
- Job N modified by another instance (optimistic lock failed) — race
- 0,
- +

JVM/Spring — Spring Batch · JobRepository 가 · JPA @Version +
OptimisticLockException CAS · Coroutines + supervisorScope (Kotlin) @Async
ThreadPoolTaskExecutor (Java) · Micrometer .

7. Active-Active 8개 노드 노드 + | 노드 노드 노드 2026.02

노드 — 노드 노드 노드 노드 노드 노드 노드 8개(blue/green/yellow/purple/orange/cyan/white/black) Active-Active 노드 노드. 8개 노드 노드 down/up 노드 traffic gap 노드 503 노드 노드 노드. 노드 노드 노드 노드·노드 노드 노드 throw 가 노드, 노드 노드 노드 노드 노드 노드 노드 노드 노드 노드(가가 ≡ 노드 → 노드 노드 → 노드 노드 노드 노드 7개 노드 노드 → 노드 retry 노드 21개 노드 노드) 노드 노드 노드 노드.

노드 — 노드 노드 노드 노드 graceful drain 노드 **5-step** 노드 노드 노드 + 노드 노드 **dict** 노드 노드 노드 노드 노드. dynamic 노드 노드 → 노드 노드 노드(노드 노드, 노드 노드 노드 → 503 노드) → 8개 노드 down + 노드 up → health check($120 \times 5s = 10min$ budget) → 노드 노드 + reload readiness 노드. 가가 노드 노드 노드 노드 0(노드 노드 노드 = 노드 노드 노드) 가 노드 노드 노드 노드 노드 노드.

노드

- 8개 노드 노드 노드 노드 노드 노드 노드 노드 노드 노드.
- 노드 노드 — 노드 노드 노드 노드 노드 노드, 가가 가 노드 0.
- 가가 노드 노드 worker 노드 노드 14 → 30 → 25 노드.

JVM/Spring 노드 — Spring Boot Actuator /health + readinessProbe / livenessProbe · Spring Cloud Gateway 노드 Reverse Proxy 노드 dynamic config · K8s rolling update 가 노드 노드 노드 노드

8. BIZ 1,000 × 7 - 2 + DB-level pagination + SQL VIEW SSOT 2026.01

— BIZ 1 1,000 7 ROAS / / , 7, (stale) , JOIN brand → ... → orders (1,000 × 7 × N) + IO ORM 4 + dynamic where + dynamic order by + dynamic LIMIT 가 .

— 「select shape N+1 JOIN sweet spot 2- batch select + SQL VIEW SSOT + IO . 1 shop (brand × brand_user × shop × brand_shop 4, LIMIT +1 has_more DB-level pagination), 2 ~20 shop_ids orders GROUP BY (140 = 20 × 7). 4(active / inactive / shop_deleted / brand_removed) 0 stale SQL VIEW API · ad-hoc · BI (single source of truth). daily upsert + 4 read.

B3. BIZ - 2 + SQL VIEW SSOT + IO

1 → 1,000 × 7 N+1 JOIN sweet spot

JOIN -

```
SELECT ...
FROM brands b JOIN brand_users bu ... JOIN shops s ... JOIN orders o ...
WHERE b.brand_id = ? AND sales_date BETWEEN ?
```

1,000 × 7 × N
→ 7,000+ × N

2-step
batch

Sweet spot - 2 batch select

① Shop (LIMIT/OFFSET DB-level)
→ 4 JOIN (brand × brand_user × shop × brand_shop)
→ ORDER BY : LIMIT %s+1 OFFSET %s (has_more)
② shop_ids orders GROUP BY
→ WHERE shop_id IN (...20 ids)
→ 140 (20 × 7) - IN lookup
→ N+1 + JOIN

API

GET /dashboard/orders
?days=7&limit=20&offset=0
1,000

1 Shop (4 JOIN)
brands brand_users shops brand_shops
WHERE b.is_active=1 ... ORDER BY ... LIMIT 21
→ shop_ids = [id1, id2, ..., id20] (+1 = has_more)

2 Orders GROUP BY (ids)
SELECT shop_id, sales_date, COUNT(order_id)
FROM orders WHERE shop_id IN (...20 ids)
GROUP BY shop_id, sales_date
→ 140 (20 × 7) - IN lookup

- 4

active :
inactive : brand_shop
shop_deleted :
brand_removed :
→ 0 , 가

SQL VIEW SSOT - stale

```
CREATE VIEW v_shop_crawling_status AS
SELECT shop_id, last_crawled_at,
CASE WHEN ... THEN 'stale' ELSE 'fresh' END AS status
FROM shops ...
```

가 API · 가 ad-hoc · BI =
SQL VIEW - SQL

IO

ROAS - : daily upsert
4 (SUCCESS / PLATFORM_FAILED / SCRAPER_FAILED / FAILED)
150 + idempotent re-fetch
→ read
→ IO 0 - p99 latency

1,000 × 7 = 7,000 → 140 (98%)

IO 0 - VIEW stale SQL - 4

B3. BIZ 2 + SQL VIEW SSOT

1

- 1,000 × 7 2 + DB-level LIMIT/OFFSET IO 0.
- VIEW stale SQL .
- 150 + idempotent re-fetch .
- 「 raw SQL

JVM/Spring 2017 — Spring Data JPA + QueryDSL 2.0.0 / dynamic order by · Native Query 2017 batch select · Hibernate @FormulaJoinTable 2017 SQL VIEW 2017 (@Subselect) · @EntityGraph 2017 N+1 2017.

□□□ **C.** □□ □□□ □□ · □ □□ □□

9. 30+ worker race | 2025.11 ~

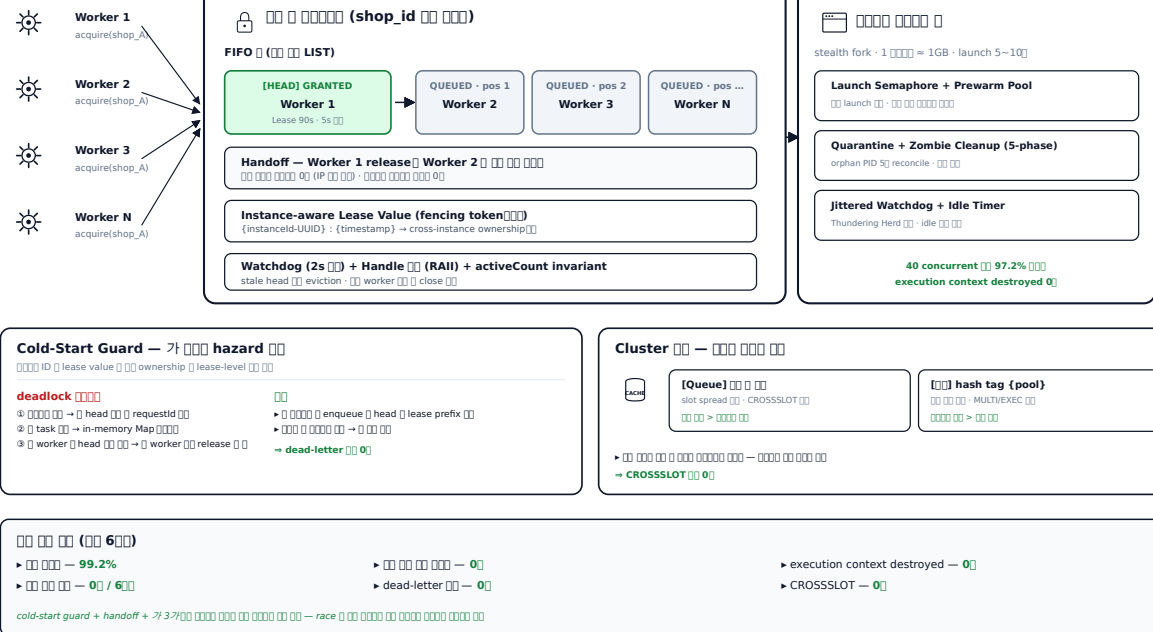
— 30+ consumer (shop_id) API(getReviews, addReply, getStores, keepAlive)가 30+ worker 가 30+ page navigation execution context destroyed, IP 1~24(1~24) mutex 1 1GB+, launch 5~10 → race N in-process Map Cluster CROSS SLOT hazard cold-start deadlock — in-memory Map head requestId worker head worker release

— 4 layer ① FIFO (LIST) ② per-request lease(90s TTL, 5s) ③ instance-aware lease value = {instanceId-UUID} : {timestamp} (Martin Kleppmann fencing token cross-instance ownership lease-level) ④ handoff pattern (release entry 가 가 cross-instance ownership lease-level) Cluster CROSS SLOT queue reputation(hash tag {pool}) attach/release Handle (RAII) activeCount invariant, pendingClose 5(immediate / defer / idle-timer / hasWaitingRequests handoff / release) worker close, release close. forceRelease(kill switch), forceTerminate(activeCount 0), evictStaleHead(2 head lease LREM) 3.

C1. 30+ worker race | 4 layer + Handoff + 3

30+ worker race | 4 layer + Handoff + 3

30+ worker race | 4 layer + Handoff + 3



`Queue`

- `Queue` `99.2%` (`6000`).
- `Queue` `0` / `6000` — `Queue` + `Idempotency-Key` `Queue`.
- `execution context destroyed` `0`.
- `Queue` `Queue` `dead-letter` `Queue` `0` — `cold-start guard` `Queue` `Queue`.
- `CROSSLOT` `0` — `hash tag` / `Queue` `Queue` `Queue` `Queue` `Queue` `Queue`.
- `Queue` `Queue` `Queue` `0`.

JVM/Spring `Queue` — `Redisson RFairLock (FIFO + lease auto-extend)` `Queue` `ShedLock` + `RLock` `Queue`
· `Kotlin Coroutines Mutex` + `withLock` `Queue` `in-process layer` · `@PreDestroy` `Queue` `graceful release` ·
`Micrometer gauge` `Queue` `Queue` `Queue`.

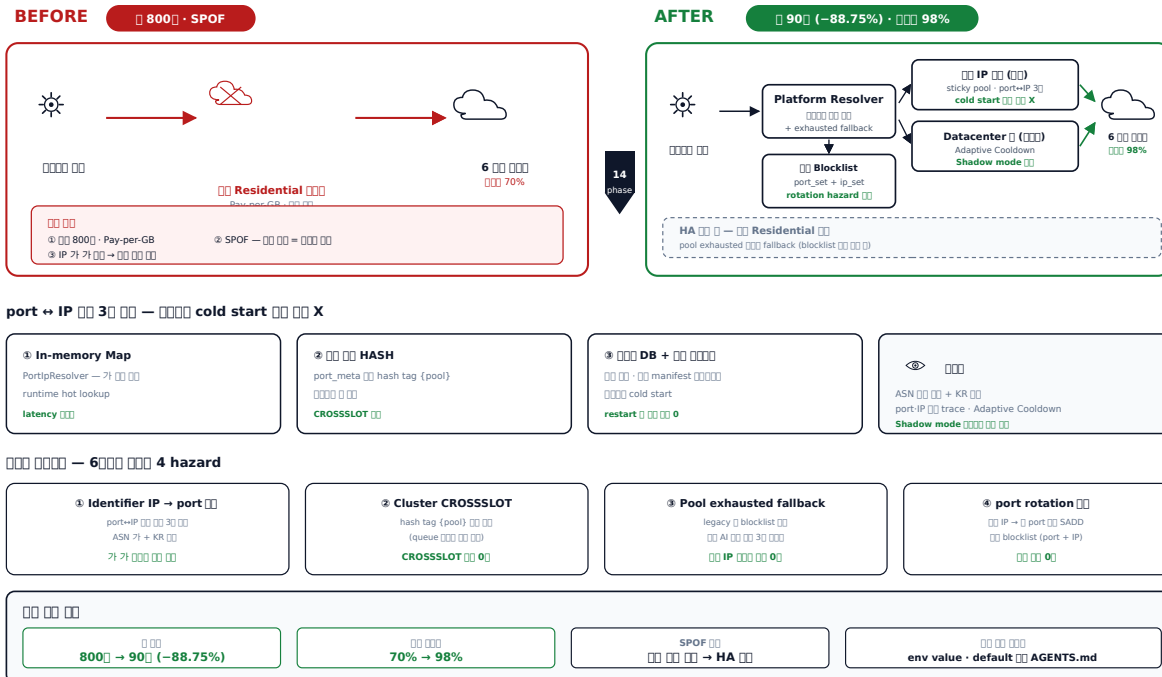
C1. C2. IP pool 문제 — 문제 풀기 + 문제 88.75% 풀기 (800 → 90) 2025.07 ~ 문제

문제 — 문제 Residential 가 문제 문제 문제 800 문제, 가 문제 문제 문제 문제 문제 SPOF 문제 문제. Datacenter 문제(90) 문제 문제 문제 datacenter IP 문제 문제 port 문제 IP 가 문제(Identifier 가 IP 문제 port 문제) 문제 문제 IP 문제 문제 문제 IP 문제 stable 문제 문제 문제 문제 문제. 가 Cluster CROSSSLOT, pool exhausted 문제 legacy fallback 문제 문제(IP 문제), 가 문제 hazard 문제 port rotation 문제(문제 IP 가 문제 문제 port 문제 mapping 문제 문제 port 가 문제 blacklist 문제 문제) 문제 6 문제 문제 문제 문제.

문제 — 14 phase phased rollout. 문제 문제 문제 문제 phase 문제 → 문제 → 문제 문제 → 문제 phase 문제 문제 cutover. 문제 문제 4가: ① port→IP 문제 3 문제 (In-memory Map + 문제 문제 HASH + 문제 DB + 문제 문제 문제 manifest) — 문제 cold start 문제 문제 문제 ② 문제 blacklist — port_set(문제 문제 port 문제 문제) + ip_set(문제 문제 IP 문제 문제), allocator 가 port 문제 문제 lookup 문제 IP 문제 → ip_set hit 문제 문제 port 문제 port_set 가 → port rotation 문제 hazard 문제 ③ Adaptive Cooldown — 5 outcome(success / block / timeout / networkError / authError / siteChange / unknownError) × consecutiveFailures × latency 가 health 문제 → ranked 문제, Shadow mode 문제 문제 문제 문제 문제 ④ Pool exhausted → legacy fallback — 문제 exhausted 문제 legacy path 문제 문제, legacy 문제 blacklist 문제 문제 IP skip(문제 10). 문제 문제 AGENTS.md 문제 문제 — env value 문제 문제 / 문제 / deprecated alias 문제 문제 default 문제 문제. fallback 문제 문제 문제 AI 문제 문제 3 follow-up 문제 문제 문제 PR 문제 문제 문제 — AI 문제 문제 문제 문제 문제 문제 문제 문제.

C2. 문제 IP 문제 문제 — 문제 문제 문제 · 문제 88.75% 문제 · 문제 70% → 98%

14 phase phased rollout — port→IP 문제 3 · Adaptive Cooldown · 문제 blacklist · pool exhausted fallback



C2. 문제 IP 문제 문제 Before/After

□□

- □ □□ **800** → **90** □ (**88.75%** □□) — □□ □□□ □□.
- □□ □□□ **70%** → **98%** — □□□ □□ □ □□ □□□□.
- □□ □□ SPOF □□(HA □□ □□ □□), □□□ □□□□ □□.
- □□ IP 가 legacy □□□ □□□□□ □□ **0**□.
- CROSSSLOT □□ **0**□ — hash tag {pool} □□ □.
- ASN □□ □□ + KR 가 + □□ □□ □□(failed outcome □□) □□.

JVM/Spring □□ — Spring Cloud Gateway + Resilience4j Retry / CircuitBreaker / Bulkhead □

□ · WebClient ExchangeFilterFunction □□ proxy resolver □□□ · Spring Configuration

@ConfigurationProperties + @Validated □ env □□ □□□.

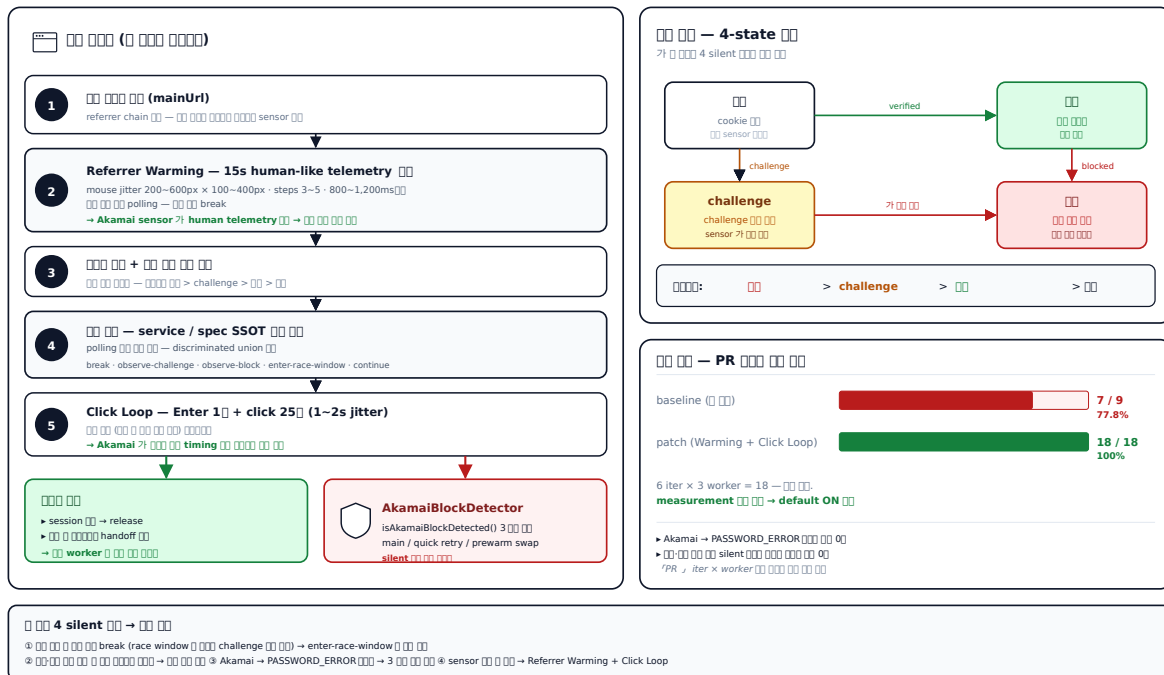
11. Akamai Bot Manager — 4-state · Referrer Warming · Click Loop (77.8% → 100%) 2026.02

— Akamai Bot Manager 가 4-state 4 silent 99% stealth fork fingerprint sensor 가 pending 403 4 가 silent ① challenge ② ③ Akamai application-level 401/403 password retry (Full Retry) ④ sensor break sensor 가 (1~2) .

— 4 가 (a) 4-state > challenge > > + (b) service spec SSOT (discriminated union — break / observe-challenge / observe-block / enter-race-window / continue) + (c) Referrer Warming — URL referrer chain, 15 human-like mouse jitter(200~600px × 100~400px × steps 3~5 × 800~1,200ms) → sensor 가 human telemetry → + (d) Click Loop — submit Enter 1 → page click 25 (1~2 jitter) + (e) 3 — Akamai helper main loginResult + Quick Retry + Prewarm Swap (set silent 가).

C3. Akamai Bot Manager — 4-state · Referrer Warming · Click Loop

4-state 4 silent 77.8% → 100% (18/18 iter 6 iter)



C3. Akamai + 4-state

•

- 77.8% → 100% (Referrer Warming, 18/18 iter, 6 iter × 3 worker, default ON).

- Akamai 関数 → PASSWORD_ERROR 関数 関数 **0**。
- 関数・関数 関数 関数 silent 「 」 関数 関数 **0**。
- PR 関数 iter × worker 関数 関数 関数 関数 関数 関数 (7/9 → 18/18)。

JVM/Spring 関数 — Spring @Component 関数 関数 関数 (sealed interface) · WebClient
 interceptor 関数 関数 関数 関数 関数 · Resilience4j TimeLimiter + Retry 関数 · Kotlin sealed class
 PollingAction 関数 discriminated union.

12. Single-Flight Coordinator — Hexagonal + Decorator 50 pp, 7 spec | 2026.01

① `platformId`는 `N`개의 `single-flight`, `coalescing`인 `Map` 80+ 개를 생성하며, 각 `Map`은 `single-flight`인 `record`을 생성하고, `record`은 `finally cleanup`을 하고, `throw` (operation sync throw `Map` record `throw`)을 하고 invariant.

Port-Adapter(Hexagonal Architecture, Alistair Cockburn) + Decorator + Strategy

OCP + spec 7가 spec describe 1:1.

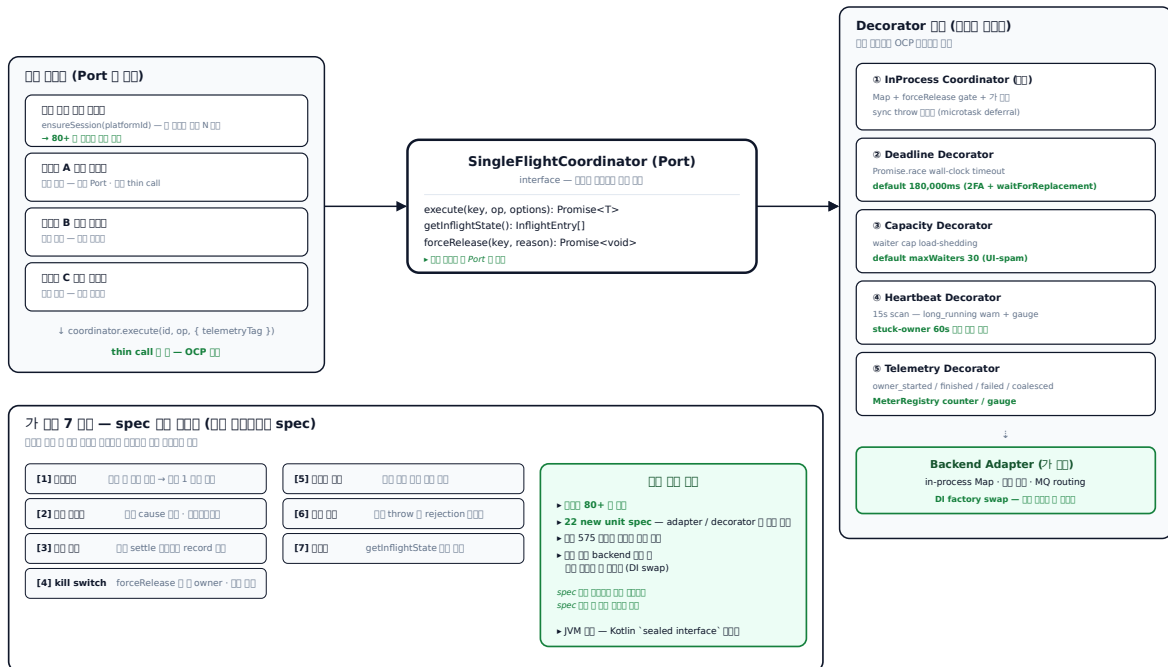
Port execute / getInflightState / forceRelease, in-process Map(forceRelease gate) + 4 (Deadline = Promise.race wall-clock timeout / Capacity = waiter cap load-shedding / Heartbeat = 15s scan long_running warn + gauge / Telemetry = owner_started/finished/failed/coalesced metrics).

this.inflight.get(key) == record, sync throw Promise.resolve().then(() => operation()) hazard.

default deadline 180,000ms(2FA + waitForReplacement), maxWaiters 30(UI-spam load-shedding), long_running warn at 60s(stuck-owner early signal), forceRelease ops kill-switch.

C4. Single-Flight Coordinator — Hexagonal Architecture + Decorator 5

가 7 spec — service 80+ → thin call



C4. Single-Flight Coordinator — Hexagonal + Decorator 5□

7 – spec (specification)

#	□□	□□
---	----	----

1	□□□□	□□ □ □□ □□□ 1□□ □□
2	□□ □□□	□□ cause □□□□ □□ (□□ □□□□ □□)
3	□□ □□	□□ settle □□□□ record □□ □□
4	kill switch	forceRelease □□ □□ □□□ □ owner, □□ □□
5	□□□ □□	□□ □□ □□ □□ □□
6	□□ □□	□□ throw □ promise rejection □□ □□□
7	□□□	getInflightState 가 entry □□ □□

□□

- □□□ □□ **80+** □ □□ — coordination + telemetry 가 thin call coordinator.execute(id, op, { telemetryTag }) □ □□.
- **22 new unit specs** — adapter/decorator □ □□ □□, □□ □□□ □□□ □□ □□.
- □□ □□ backend 가 □ □□□ □□ □ □□□□ DI factory □ swap.
- 「 □□□□□ spec □□ □□□□ □□□□ □□ □□□ □□ □□□□□□ □□ □□□ □□□.

JVM/Spring □□ — Spring @Component + @Qualifier □ backend □□□ swap · Kotlin Coroutines + Deferred □ in-process single-flight · Resilience4j Bulkhead.semaphore + TimeLimiter □ deadline · MeterRegistry counter / gauge □ telemetry.

13. 1000개 리뷰 — 600개 리뷰 + 가 0.5~1.0 OR | 2026.01 ~ 12월

1000개 리뷰 — 600개 리뷰 + 가 0.5~1.0. ① 1000개 500개 리뷰 sentiment 가 0.5~1.0 30% 가 50 + 가 0.5 ② "ㅋㅋㅋㅋㅋㅋ" 1000개 리뷰 1000개 리뷰 ML 0.5~1.0 0.5~1.0, AI 1000개 "가 0.5~1.0 1000개 리뷰 1000개 리뷰. 가 0.5~1.0 1000개 리뷰 1000개 리뷰 — 1000개 "0.5~1.0" 0.5~1.0, 50 "0.5~1.0" 0.5~1.0.

1000개 리뷰 + 1000개 리뷰 1000개 리뷰 1000개 리뷰 if-else 1000개 리뷰 600개 리뷰 1000개 리뷰 1000개 리뷰 ML 1000개 리뷰. 1000개 리뷰 shrink (Bell) 1000개 리뷰 1000개 리뷰 (Polarity Amplification) 1000개 리뷰 1000개 리뷰 + tanh ±35 sigmoidal squashing 가 1000개 리뷰 1000개 리뷰 1000개 리뷰.

1000개 리뷰 600개

1000개	1000개
1000개 (top word share)	top_word_count / total_tokens
Bigram / Trigram 1000개	most_common_ngram / total_ngrams
1000개	$-\sum p \log_2 p$ (1000개)
1000개	unique_chars / length (40개 1000개)
zlib 1000개	len(compressed) / len(text)
1000개 / 1000개 4+ 1000개	1000개 1000개

1000개 리뷰 1000개 리뷰 2.0 1000개 noise. 1000개 리뷰 Bell 1000개 리뷰 shrink + 1000개 리뷰 weight 1000개 리뷰 tanh ±35 box 1000개 squash. 1000개 리뷰 rating ≤ critical_rating 1000개 리뷰 OR 1000개 리뷰 50 + score in (min~max) + len ≥ 1000개 scoring-based 1000개. 1000개 BIZ(1000개 `send\_negative\_alarms` + `critical\_review\_rating` 1000개) 1000개 User(`NegativeReviewAlarm.is\_active`) 1000개 리뷰, 1000개 리뷰 1000개 리뷰 1000개 리뷰 = 1000개 리뷰 1000개 리뷰 1000개. 1000개 리뷰 SCORE_VERSION 1000개 PK 1000개 리뷰 review_id 1000개 리뷰 1000개 리뷰 → v2 1000개 리뷰 v1 1000개 리뷰 1000개 리뷰.

1000개

- AI 1000개 리뷰 — existing_scores_map pre-fetch 1000개 리뷰 1000개 리뷰 + 1000개 리뷰 1000개 리뷰 ML 1000개 skip.
- 50개 리뷰 1000개 리뷰 — 1000개 + 1000개 OR 1000개 1000개 30% 가 0.5 ≠ 1000개 리뷰.
- 1000개 리뷰 1000개 BIZ 30 + User 30 = 60 1000개 리뷰 1000개.
- 1000개 리뷰 1000개 리뷰 — 1000개 (1000개·1000개) + 1000개 리뷰 1000개 Enum 1000개 리뷰.

JVM/Spring 1000개 — Spring Batch ItemProcessor 1000개 리뷰 · RestTemplate / WebClient 1000개 AI Score API · Bean Validation 1000개 리뷰 1000개 · ApplicationEventPublisher + @TransactionalEventListener 1000개 리뷰 1000개.

14. SafeCron – cron の安全な実行 (14 頁) 2025.09 ~

00 — 00 0000 0000 00 @Cron 0 00 0000000 00 00. 00 00 000 000 N0 0000000 N0 → 00 PG API
 rate-limit 00, 00 000 cron N0 → 0000 00 000 N0, 000 000 00 N0 → 00 SaaS rate-limit + 00. 00
 cron 00 0 00 000 00 0000 000000000 0000, 0 00 00 0 0000 0000 00 00 000 cron 0 00 0 00 000.

00 — 000000 + **DI** 00 00 00 0 0 0000000 00 0 + Slack 0000 + 0000 000 000. 0000000 000 00 000 00
 가 00000000 0000 ModuleRef 00 0000 000 0 000000 0000 00 00 0000 lazy 000000. 0 TTL 00 000
 OOM / SIGKILL 000 00 cron 00 00. 0000 Slack 0000 00 — start / complete (\${duration}ms) /
 error - \${message} + 00 trace block — 가 0 000000. slackAlert: false 0 00000 cron 00 00,
 slackWebhookUrl 0 cron 0 00 00 0000. CRON_ENABLED 000000 0000 0000 000000 00 noop → 0000
 00 00 → QA 00.

11

- **구독 14** (subscriptions, **구독**, **구독** SaaS **구독**, magic-link, mixpanel, log-cleanup **구독**).
- **구독** 5~10**구독** **구독** **구독** → **구독** **100****구독** **구독** + cron 가 **구독** **구독** **구독**.
- **구독** **구독** **구독** **구독** cron **구독** — QA **구독** **구독** **구독** **구독**.
- **구독** **구독** **구독** — DI **구독** + Lock acquire/release/TTL + Slack **구독** + **구독** **구독** **구독** **구독** **구독**.

JVM/Spring ☐ — Spring @Scheduled + ShedLock (Redis/JDBC backend) — ☐ ☐☐☐ cron ☐
☐ ☐☐ ☐☐ · @ConditionalOnProperty ☐ ☐☐☐ ☐☐ · Spring AOP @Around ☐ Slack ☐☐☐.

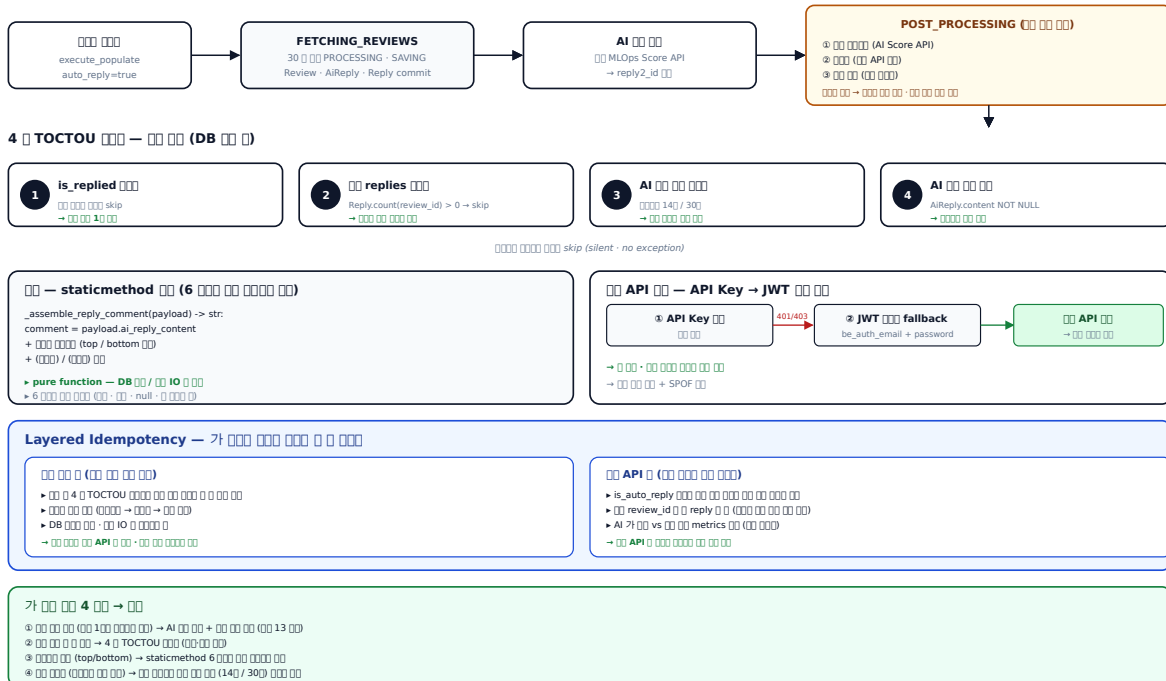
15. 1000 1000 1000000 — 100 → AI → 4 TOCTOU 1000 → API Key/JWT 100 100 100 2026.03

100 — 「10000 1000 AI 가」 10000 4가. ① 100 100 100 100 100(100 100 "1000 1000 100" 100 100 = 100) ② 100 100 100 100 ③ 1000 10000 100(100 10000) AI 100 100 / 100 100 100? ④ 100 1000(10000 "500" / "400 1000" 100 100). 가 DB 100 1000 600 10000 1000 100 API 100 100 100 100 10000.

100 — 10000 100 100 + **API Key → JWT** 100 100 **fallback** + **TOCTOU 4** 1000 100 300 1000. payload 1000 100 DB 100, 1000 staticmethod, 1000 async 100 300 1000 100 100 10000 DB 1000 100 100(Spring @Transactional 100 1000 100 100). 4 TOCTOU 10000 ① is_replied 1000 ② 100 replies 1000 ③ AI 100 100 1000 ④ AI 100 100 100 — 10000 10000 1000 skip. 1000 staticmethod 100 1000 6가 1000 100 10000 100. 100 API 100 100 API Key 100 → 401/403 100 100 JWT 1000000 100, 100 100 1000 1000 100 100. 1000 100 100 100 — 100 10000 → 1000 → 100 100(가 1000, 1000 1000 10000 100 10000 100 1000 100 100).

D1. 100 100 100 1000000 — 100 → AI → 4 TOCTOU 1000 → 100 100 100

10000 100 100 + API Key → JWT fallback + Layered Idempotency 100 300 1000



D1. 100 100 100 1000000 — TOCTOU 4 1000 + 100 100

Layered Idempotency 100 100

- 100 100 100: 100 100 4 TOCTOU 1000 100 100 100 100 100(100 100 100 100)
- 100 API 100: 10000 1000 100 100 1000 100 100 100 100(100 review_id 100 reply 100)
- 100 1000 100 **API** 100 100, 100 **API** 100 1000 10000 100 100 100 — 10000 1000 1000 100 100 100 100 100 100
- 1000 100 100 10000 100 → **AI** 100 → 100 100 100.
- 100 100 4 가 1000 100 / 1000 100 100, 100 100 staticmethod 1000 6가 1000 100 1000 100.
- API Key → JWT fallback 100 100 100 1000 1000 100 100.

- AI `metrics` vs `metrics` 가 「 — AI `metrics` vs 「 」 `metrics`.

JVM/Spring `metrics` — Spring `@Transactional(readOnly=true)` `metrics` + `RestClient` `metrics` `metrics` `metrics` · Spring Security `OAuth2AuthorizedClientManager` `API Key` → `JWT fallback` · `Resilience4j` `Retry` + `CircuitBreaker` `metrics`.

JVM 애플리케이션 — Node.js 애플리케이션 Kotlin/Spring 애플리케이션 + ADR

5개 애플리케이션 Node.js 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 Java/Kotlin/Spring 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션, 애플리케이션 애플리케이션 ADR 애플리케이션 애플리케이션 애플리케이션. 애플리케이션 **ADR 49**, 애플리케이션 **47**, 애플리케이션 애플리케이션 **17** 애플리케이션.

Node 애플리케이션 애플리케이션 ↔ Kotlin/Spring 애플리케이션 애플리케이션

Node 애플리케이션	Kotlin/Spring 애플리케이션
애플리케이션 애플리케이션 SETNX + Lua 애플리케이션 애플리케이션	Redisson watchdog + Pub/Sub 4개 애플리케이션
Idempotency Key + 애플리케이션 + 애플리케이션 + reconciliation	ADR-006 애플리케이션 애플리케이션 4개 + EXP-09b 9 애플리케이션
애플리케이션 애플리케이션 FIFO + lease TTL	Coroutines + supervisorScope + Single-Flight 5 invariants
Custom Error Strategy 5개 + 9개 애플리케이션	Resilience4j 5개 + 9개 애플리케이션 애플리케이션
애플리케이션 애플리케이션 PID Registry (OOM 애플리케이션)	JVM Heap / GC 애플리케이션 G1GC Region + Humongous
애플리케이션 애플리케이션 classic queue (애플리케이션 5개 애플리케이션 애플리케이션)	Kafka 애플리케이션 애플리케이션 + Outbox Relay

애플리케이션 애플리케이션

- 애플리케이션·애플리케이션 애플리케이션 애플리케이션 (Java / Spring Boot / JPA) — 애플리케이션 애플리케이션 4개 애플리케이션, 애플리케이션 애플리케이션 **4개** 애플리케이션(애플리케이션 · 애플리케이션 · GET_LOCK · Redisson), DB InnoDB RR vs ANSI RR 애플리케이션 애플리케이션 애플리케이션, 애플리케이션 애플리케이션 애플리케이션 9 애플리케이션 애플리케이션, No-offset Pagination.
- 애플리케이션 애플리케이션 애플리케이션 애플리케이션 (Spring Batch + Kafka) — 애플리케이션 애플리케이션 1개 애플리케이션 5개 애플리케이션(replay 애플리케이션, prefetch HoL, x-overflow, publisher confirm 애플리케이션, DLQ 애플리케이션 애플리케이션) 애플리케이션 애플리케이션 Kafka 애플리케이션 애플리케이션 ADR 애플리케이션. Outbox Relay 애플리케이션 애플리케이션 polling vs CDC 애플리케이션 애플리케이션, Spring Batch Reader 4개 애플리케이션.
- 애플리케이션 애플리케이션 애플리케이션 (Kotlin / Coroutines / Resilience4j) — 애플리케이션 애플리케이션 Kotlin 애플리케이션 애플리케이션. **Single-Flight** 애플리케이션 애플리케이션 애플리케이션(promise sharing, sync throw normalization, force release, deadline, capacity), 9 애플리케이션 애플리케이션 애플리케이션 애플리케이션, Resilience4j 5 애플리케이션(CircuitBreaker, Retry, Bulkhead, RateLimiter, TimeLimiter) 애플리케이션.

📄 — 📄📄📄(I-BRICKS) | | 📄 2021.05 ~ 2024.11

📄📄 📄📄 📄 📄 📄📄📄 📄·📄📄 📄📄, 📄📄 📄📄📄📄, 📄📄 📄📄 📄📄📄📄.

- 📄 📄📄📄 📄 📄·📄 — Elasticsearch + Logstash 📄 RDB → ES 📄📄📄 📄, 📄📄📄 📄 📄·📄📄📄 📄
가 📄 📄📄📄 📄·📄📄 📄📄 📄 📄📄📄📄 📄📄 📄.
- **EBS** 📄 📄📄 📄📄 📄📄📄 — 📄 📄📄 📄 📄📄 Kafka + Apache Nifi + Elasticsearch 📄📄📄 📄. 📄
📄 📄 **50%** 📄, 📄📄 📄📄 2가 📄📄📄📄📄.
- 📄📄 📄 📄📄 — React / Redux / SCSS 📄 📄📄 UI, KWCAG 2.1 📄 📄📄 📄.

00 — 0000 000 00 00 00

00 50 000 000 00 000 000000 000000.

00·0000

- 00 9 0000 0000 — 00 / 00 / 00·000 / 00(00·00) / 00 / 0000 00 / 000 00(00·00) / 0000 → 00 000 00.
- 000 00 00 000 00 — 000 DB 00 00 0 + SELECT FOR UPDATE 00 00 + 00 00 Pub/Sub 00 00 + 00 000 00 0000 **TPS 100 → 1,000+**, 00 00 **50 → 100** 00, 00 000 **1%** 00 00.

0 · 000 · 000

- 000 0 DLX 00 application-level retry — setTimeout + republishMessage 0 hot loop 00, 30s / 5m / 30m 000 backoff. Phase 2 00 DLX 000 00 000000 00 00.
- 00 00 IntegrityError-aware upsert + Lock-timeout 00 000 — Duplicate 0 break, Lock timeout 0 1s / 2s / 3s 00 backoff, 00 00 raw SQL ON DUPLICATE KEY UPDATE 00.

00 000 · 00 00

- 00 API quarantine — DB-backed state machine(OK / QUARANTINED) + 12h probe 00 00 + DB↔00 00 mismatch 00.
- 00 0000 00 00 + 00 00 0000 — 00 00(00 000 000 ++) / 00 API 00 200, 00 30 00 0000 + 00 0 가 00 00.
- 000 00 00 000 — 0000 180 + 110+ 000 BasePlatformException → 0000 000 → classifier helper 0 3 layer 0. instanceof + name + code 30 000 000 00 00.
- Idempotent Reply Pipeline — Lease + Completion Cache 00 + 40 ErrorClassifier(BLOCKED_SUSPECTED / TRANSIENT / PERMANENT / SYSTEM) + ACK/NACK 0000 000. 000 PERMANENT(unknown 00 00 retry 00).
- Adaptive Traffic Controller + Launch Budget — Lua Token Bucket + Circuit Breaker(SOFT_OPEN / HALF_OPEN) + jittered watchdog 00 Thundering Herd 00.

00 00 · 000

- 0000 0000 0 — Launch Semaphore + Prewarm Pool + Quarantine + 5-phase Zombie Cleanup + Jittered Watchdog 50 00 00. 40 concurrent 00 **97.2%** 000.
- OOM 00 chunk 00 — 00 00 00 cron 0 find 000 000 CHUNK_SIZE=2000 while 00 + lookup map 10 0000 00.
- 000 0000 000 — 000 0000 300 00 + Semaphore 30 + 00 DB 00 + 000 gc + 300 00 00000 00 0 000. 000 0000 00 gc 000 000.

0000 · 00

- 00 0000 0000000 — 00 00000000 00 00 Hash + ZSet(000 / 000 000) + TTL 000 00 (RUNNING 6h / COMPLETED 1h / FAILED 2h)0 00. 00 000 0 orphaned job 00 000 (Semaphore 10 000 00).
- 2400 00 000 + Config 0 RUNNING 10 — 2400 0 000 DB 0 00 00 ±50 000 dedupe 0 00 00, Config 0 RUNNING 10 00 + 00 00 Semaphore(200).

- `backfill` — `fetch in-memory set` `backfill`, `backfill` `backfill` `backfill`.
- `backfill` `backfill` `backfill`.
- `backfill` `backfill` `SaaS` `backfill` `backfill` — `backfill` `backfill` `backfill` `backfill` `backfill` `backfill`, `backfill` `backfill` `backfill` `backfill`.
- `APM Span Tagging` — `tagDatadogSpan` `backfill` `backfill` `backfill`(`backfill` `backfill`, `backfill` `amount mismatch`) `backfill`.

☐☐ ☐☐

☐☐	☐☐ ☐☐
☐☐	TypeScript, JavaScript, Kotlin , Java , Python
☐☐☐☐☐	NestJS, Node.js, Spring Boot , Spring Batch , FastAPI
ORM	TypeORM, Python ORM, JPA / Hibernate , APScheduler
☐☐☐☐☐☐	☐☐☐ DB (Cluster, Replica) — MySQL ☐☐ / PostgreSQL / Oracle
☐☐ · ☐	☐☐ ☐☐ (Cluster ☐☐), Lua eval, Token Bucket, Redisson
☐☐☐	☐☐☐ ☐ (Quorum Queue, DLX, classic queue ☐☐ ☐ ☐), Kafka
☐☐	Elasticsearch, Logstash, Apache Nifi
☐☐ ☐☐☐ ☐☐	Resilience4j , Custom Retry / Timeout, Single-Flight (Port-Adapter), Circuit Breaker (SOFT_OPEN / HALF_OPEN), Token Bucket
☐☐☐☐ ☐☐☐☐	stealth fork ☐☐ ☐☐☐☐ ☐, ☐☐ ☐☐ 5☐
☐ ☐☐ ☐☐	Akamai · Cloudflare ☐☐ — ☐☐ ☐☐ 4-state ☐☐, ☐☐ ☐☐ polling, Referrer Warming, Click Loop jitter
☐☐ · ☐☐	Prometheus, Grafana, APM (span tagging) , nGrinder, JFR
☐☐☐	Jest, Testcontainers , JUnit 5 , Kotest , Pytest, "☐☐ spec ☐☐ ☐☐☐" ☐☐☐
☐☐☐	Docker, Docker Compose × 8 color, ☐☐ ☐☐☐ ☐☐, ☐ ☐☐☐ ☐☐☐☐☐☐☐, ☐☐ Cloud, Jenkins, GitHub Actions
☐☐☐☐☐☐	Flyway, Liquibase ☐☐
☐☐	Slack Webhook (Block Kit), ☐☐☐ OAuth2 + ☐☐☐ ☐☐ ☐☐☐
AI / ML	☐☐ MLOps Score API + 6☐☐ ☐☐☐ ☐☐ (Shannon entropy, zlib ☐☐☐, n-gram, emoji run) + 가 ☐ ☐☐ (Bell + Polarity Amplification + tanh box)
☐☐ ☐☐☐ ☐☐	Redlock(Martin Kleppmann) ABA ☐☐ fencing token, Hexagonal Architecture(Port-Adapter), CQRS, Saga, Outbox, CAS, Bulkhead, Circuit Breaker, Single-Flight



F-Lab Java Backend Mentoring | 2024.01 ~ 2024.07

Meta 000 000 000 00 00. 0000 00, 0000 00, 00 0000 0000 000 00, CQRS, 00 000000 00 00.

□ □ □ □ □ □ □ □ □ □ □ □ □